

1- Crea un programa en C con las siguientes funciones:

C/C++

```
void swap(int *a, int *b)
```

- Una función que intercambia dos enteros pasados por referencia.

C/C++

```
void reverseArray(int arreglo[], int tamaño)
```

- Una función que invierte un arreglo de enteros pasado por referencia.

C/C++

```
void printArray(int arreglo[], int tamaño)
```

- Una función que imprime los elementos de un arreglo de enteros.

En la función main():

- Declara un puntero a int llamado arreglo e inicialízalo como NULL. Este puntero se utilizará para el arreglo.
- Inicializa tamaño como 5, que representará el tamaño inicial del arreglo.
- Valores del arreglo: 10 20 30 40 50
- Utiliza malloc para asignar memoria dinámica al arreglo con el tamaño inicial
- Inicializa los elementos del arreglo con valores de tu elección.
- Imprime el arreglo original.
- Cambia el tamaño del arreglo a 7
- Inicializa los elementos adicionales del arreglo según sea necesario.
- Imprime el arreglo después de cambiar el tamaño.
- Llama a la función reverseArray para invertir el arreglo.
- Imprime el arreglo invertido.
- Llama a la función swap para intercambiar dos elementos en el arreglo (índice 1 x índice 3).
- Imprime el arreglo después de la operación de intercambio.

Restricciones:

Utiliza el paso por referencia para todos los parámetros de las funciones .
Maneja adecuadamente la asignación y liberación de memoria dinámica.

Salida de Ejemplo:

Arreglo Original: 10 20 30 40 50

Arreglo después de cambiar el tamaño: 10 20 30 40 50 60 70

Arreglo Invertido: 70 60 50 40 30 20 10

Arreglo después del intercambio: 70 40 50 60 30 20 10

C/C++

```
int main() {

    int *arreglo = NULL; // Puntero que representará el arreglo dinámico

    int tamaño = 5;      // Tamaño inicial del arreglo

    /* Muy importante entender las primeras dos líneas antes de seguir

        malloc reserva memoria dinámica para 5 enteros.

        El resultado se guarda en el puntero arreglo.

    */

    arreglo = (int *)malloc(tamaño * sizeof(int));

    // Verifica si malloc pudo asignar memoria correctamente

    if (arreglo == NULL) {

        printf("Error al asignar memoria.\n");

        return 1;

    }

    // Inicialización de los primeros 5 elementos del arreglo

    arreglo[0] = 10;

    arreglo[1] = 20;

    arreglo[2] = 30;

    arreglo[3] = 40;
```

```

arreglo[4] = 50;

// Imprime el arreglo original
printf("Arreglo Original: ");
printArray(arreglo, tamaño);

// Cambiamos el tamaño lógico del arreglo de 5 a 7
tamaño = 7;

/*
    realloc intenta cambiar el tamaño del bloque de memoria.

    Se usa un puntero temporal para no perder la referencia original
    si realloc falla.
*/

int *temp = (int *)realloc(arreglo, tamaño * sizeof(int));

// Verifica si realloc pudo redimensionar el arreglo correctamente
if (temp == NULL) {
    printf("Error al cambiar el tamaño del arreglo.\n");
    free(arreglo); // Libera la memoria original antes de terminar
    return 1;
}

// Si realloc fue exitoso, actualizamos el puntero principal
arreglo = temp;

// Inicializamos los nuevos elementos agregados al arreglo
arreglo[5] = 60;
arreglo[6] = 70;

// Imprime el arreglo después de cambiar el tamaño

```

```

printf("Arreglo después de cambiar el tamaño: ");

printArray(arreglo, tamaño);

// Invierte el arreglo completo
reverseArray(arreglo, tamaño);

// Imprime el arreglo invertido
printf("Arreglo Invertido: ");

printArray(arreglo, tamaño);

/*
    Intercambia los elementos en índice 1 e índice 3.

    Después de invertir:

    índice 1 = 60

    índice 3 = 40

    Luego del intercambio:

    índice 1 = 40

    índice 3 = 60
*/

swap(&arreglo[1], &arreglo[3]);

// Imprime el arreglo después del intercambio
printf("Arreglo después del intercambio: ");

printArray(arreglo, tamaño);

// Libera la memoria dinámica usada por el arreglo
free(arreglo);

// Evita que el puntero quede apuntando a memoria liberada
arreglo = NULL;

```

```
    return 0;  
}
```

2-En este reto de programación, debes implementar operaciones básicas en una lista basada en arreglos utilizando punteros y referencias de memoria en el lenguaje de programación C. Debes crear funciones para realizar operaciones comunes en una lista de arreglos, como inserción, eliminación, recorrido y encontrar la longitud de la lista.

Instrucciones:

Implementa una estructura básica para la lista basada en arreglos con los siguientes componentes:

Un arreglo de enteros llamado `listArray` para almacenar los valores de datos de la lista.

Una variable entera llamada `listSize` para llevar un registro del número actual de elementos en la lista.

Una constante entera llamada `MAX_SIZE` (en este caso 100) para especificar la capacidad máxima de la lista de arreglos.

Implementa las siguientes funciones:

a.

```
C/C++  
void initArrayList(int* listArray, int* listSize)
```

Inicializa la lista de arreglos estableciendo `listSize` en 0

b.

C/C++

```
void insertAtBeginning(int* listArray, int* listSize, int data)
```

Inserta un nuevo elemento con el valor de datos proporcionado al principio de la lista de arreglos.

c.

C/C++

```
void insertAtEnd(int* listArray, int* listSize, int data)
```

Inserta un nuevo elemento con el valor de datos proporcionado al final de la lista de arreglos.

d.

C/C++

```
void insertAfterIndex(int* listArray, int* listSize, int data, int index)
```

Inserta un nuevo elemento con el valor de datos proporcionado después del elemento en el índice proporcionado en la lista de arreglos.

e.

C/C++

```
void deleteAtIndex(int* listArray, int* listSize, int index)
```

Elimina el elemento en el índice proporcionado de la lista de arreglos.

f.

C/C++

```
int findLength(int* listSize)
```

Devuelve la longitud de la lista de arreglos (número de elementos).

g.

C/C++

```
void printArrayList(int* listArray, int* listSize)
```

Imprime los valores de datos de todos los elementos en la lista de arreglos.

Debes usar punteros y referencias de memoria para manipular la lista de arreglos. No utilices arreglos dinámicos o estructuras de datos incorporadas.

Maneja adecuadamente los casos especiales, como cuando la lista está llena o vacía, o cuando se intenta eliminar un elemento desde un índice inválido. (imprime un mensaje que indica la ocurrencia del caso especial)

El Output debe ser igual a:

10 -> 20 -> 30
5 -> 10 -> 20 -> 30
5 -> 10 -> 20 -> 25 -> 30
5 -> 20 -> 25 -> 30
Length of the array list: 4

10 -> 20 -> 30
5 -> 10 -> 20 -> 30
5 -> 10 -> 20 -> 25 -> 30
5 -> 20 -> 25 -> 30
Length of the array list: 4

```
#include <stdio.h>

#define MAX_SIZE 100

void initArrayList(int* listArray, int* listSize) {
    //code goes here
}

void insertAtBeginning(int* listArray, int* listSize, int data) {
    //code goes here
}

void insertAtEnd(int* listArray, int* listSize, int data) {
    //code goes here
}

void insertAfterIndex(int* listArray, int* listSize, int data, int index) {
    //code goes here
}

void deleteAtIndex(int* listArray, int* listSize, int index) {
    //code goes here
}
```

```

}

int findLength(int* listSize) {
    //code goes here
}

void printArrayList(int* listArray, int* listSize) {
    //code goes here
}

int main() {
    int listArray[MAX_SIZE];
    int listSize = 0;

    initArrayList(listArray, &listSize);

    insertAtEnd(listArray, &listSize, 10);
    insertAtEnd(listArray, &listSize, 20);
    insertAtEnd(listArray, &listSize, 30);
    printArrayList(listArray, &listSize); // Output: 10 -> 20 -> 30

    insertAtBeginning(listArray, &listSize, 5);
    printArrayList(listArray, &listSize); // Output: 5 -> 10 -> 20 -> 30

    insertAfterIndex(listArray, &listSize, 25, 2);
    printArrayList(listArray, &listSize); // Output: 5 -> 10 -> 20 -> 25 -> 30

    deleteAtIndex(listArray, &listSize, 1);
    printArrayList(listArray, &listSize); // Output: 5 -> 20 -> 25 -> 30

    int length = findLength(&listSize);
    printf("Length of the array list: %d\n", length); // Output: Length of the
array list: 4

    return 0;
}

```